



Язык программирования C++

Лекция 2

STL

Библиотека стандартных шаблонов (**STL**) — набор согласованных обобщённых алгоритмов, контейнеров, средств доступа к их содержимому и различных вспомогательных функций в C++

Algorithms

Containers

Functors

Iterators

ПОСЛЕДОВАТЕЛЬНЫЕ КОНТЕЙНЕРЫ

array

vector

deque

forward_list

list

подключить
соответствующую
библиотеку с помощью
директивы #include



описать тип элементов

ARRAY

```
#include<array>
```

```
array<int, 6> ar = { 0, 1, 2, 3, 4, 5 };  
for (int i = 0; i < 6; i++)  
    cout << ar.at(i) << " ";  
  
cout << get<2>(ar) << " " << get<3>(ar);  
  
for (int i = 0; i < 6; i++)  
    cout << ar[i] << " ";
```

```
int& a = ar.front();  
int& b = ar.back();
```

```
ar.size()
```

```
array<int, 6> ar1 = { 6, 7, 8, 9, 10, 11 };  
ar.swap(ar1);
```

```
array<int, 0> ar3;  
ar3.empty() ? cout << "Array empty" :  
             cout << "Array not empty";
```

```
array<int, 6> ar2;  
ar2.fill(0);
```

VECTOR

begin()

end()

rbegin()

rend()

cbegin()

cend()

crbegin()

crend()

```
vector<int> p;  
for (int i = 1; i <= 10; i++)  
    p.push_back(i * 10);  
cout << p[2];  
cout << p.at(4);  
cout << p.front();  
cout << p.back();  
int* pos = p.data();  
cout << *pos;
```

```
#include <vector>
```

assign()

push_back()

pop_back()

insert()

erase()

swap()

clear()

emplace()

emplace_back()

size()

max_size()

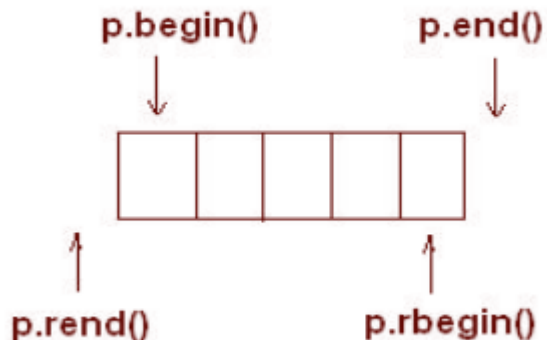
capacity()

resize(n)

empty()

shrink_to_fit()

reserve()



DEQUE

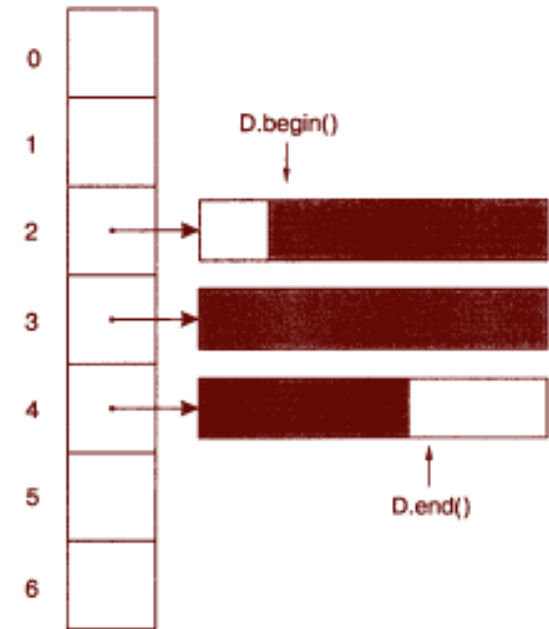
```
#include <deque>
```

```
void showdq(deque<int> d)
{
    deque<int>::iterator it;
    for (it = d.begin(); it != d.end(); ++it)
        cout << '\t' << *it;
    cout << '\n';
}
```

```
deque<int> d;
d.push_back(10);
d.push_front(20);
d.push_back(30);
d.push_front(15);
```

```
cout << d.at(2);
cout << d.front();
cout << d.back();
```

```
d.pop_front();
d.pop_back();
showdq(d);
```



FORWARD LIST

```
#include <forward_list>
```

```
flist.clear();
```

```
forward_list<int> flist;  
flist1.assign({ 1, 2, 3 });
```

```
flist.remove_if([](int x) { return x > 20; });
```

```
flist.push_front(4);  
flist.emplace_front(5);  
flist.pop_front();
```

```
forward_list<int> flist1 = { 10, 20, 30 };  
forward_list<int> flist2 = { 40, 50, 60 };  
flist2.splice_after(flist2.begin(), flist1);  
for (int& c : flist2)  
    cout << c << " ";  
cout << endl;
```

```
40 10 20 30 50 60
```

```
forward_list<int> flist = { 10, 20, 30 };  
forward_list<int>::iterator ptr;  
ptr = flist.insert_after(flist.begin(), { 1, 2, 3 });  
ptr = flist.emplace_after(ptr, 2);  
ptr = flist.erase_after(ptr);  
ptr = flist.erase_after(flist.begin(), flist.end());
```

```
10 1 2 3 20 30  
10 1 2 3 2 20 30  
10 1 2 3 2 30  
10
```

LIST

```
#include <list>
```

```
list<int> l1, l2;
for (int i = 0; i < 5; ++i) {
    l1.push_back(i * 2);
    l2.push_front(i * 3);
}
showlist(l1);
showlist(l2);
cout << l1.front() << endl;
cout << l1.back() << endl;
l1.pop_front();
showlist(l1);
l2.pop_back();
showlist(l2);
l1.reverse();
showlist(l1);
l2.sort();
showlist(l2);
```

```
void showlist(list<int> l)
{
    list<int>::iterator it;
    for (it = l.begin(); it != l.end(); ++it)
        cout << '\t' << *it;
    cout << '\n';
}
```

	0	2	4	6	8
	12	9	6	3	0
0					
8					
	2	4	6	8	
	12	9	6	3	
	8	6	4	2	
	3	6	9	12	

СЦЕПКА СПИСКОВ

служит для перемещения элементов из одного списка в другой без перераспределения памяти, только за счет изменения указателей

`splice(p, L2)`

Вводим в первый список: 1 2 3 4 5
Вводим во второй список: 6 7 8 9 0
Результат:
1 2 3 6 7 8 9 0 4 5
Список пуст

```
int a;
list<int> L1, L2;
list<int>::iterator p;
for (int i = 1; i <= 5; i++)
{
    cin >> a;
    L1.push_back(a);
}
for (int i = 1; i <= 5; i++)
{
    cin >> a;
    L2.push_back(a);
}
p = L1.begin();
for (int i = 1; i <= 3; i++) //переходим к 4-му элементу списка L1
    p = ++p;
L1.splice(p, L2);
for (p = L1.begin(); p != L1.end(); p++)
{
    cout << *p << " ";
}
cout << "\n";
if (L2.empty()) cout << "Список пуст";
else
    for (p = L2.begin(); p != L2.end(); p++)
    {
        cout << *p << " ";
    }
cout << "\n";
```

СЦЕПКА СПИСКОВ

```
splice(p1, L2, p2);
```

```
splice(p1, L2, q1, q2);
```

```
p2 = L1.begin();  
for (int i = 1; i <= 3; i++)    // Ищем 4-ый элемент  
    p2 = ++p2;  
p1 = L1.begin();  
L1.splice(p1, L1, p2);    // Переносим 4-ый элемент в начало списка  
for (p1 = L1.begin(); p1 != L1.end(); p1++) {  
    cout << *p1 << " ";  
}
```

Вводим: 1 2 3 4 5

Результат:

4 1 2 3 5

ИТЕРАТОРЫ

имя контейнера <тип данных> :: iterator имя итератора

Random-access

Bidirectional

Forward

Input | Output

vector

• Random-access

list

• Bidirectional

deque

• Random-access

map

• Bidirectional

multimap

• Bidirectional

set

• Bidirectional

multiset

• Bidirectional

stack

• No iterator supported

queue

• No iterator supported

priority-queue

• No iterator supported

ОПЕРАЦИИ С ИТЕРАТОРАМИ

`*iter`: получение элемента, на который указывает итератор

`++iter`: перемещение итератора вперед для обращения к следующему элементу

`--iter`: перемещение итератора назад для обращения к предыдущему элементу

`iter1 == iter2`: два итератора равны, если они указывают на один и тот же элемент

`iter1 != iter2`: два итератора не равны, если они указывают на разные элементы

ОПЕРАЦИИ С ИТЕРАТОРАМИ

ITERATORS	PROPERTIES				
	ACCESS	READ	WRITE	ITERATE	COMPARE
INPUT	->	= *i		++	==, !=
OUTPUT			*i =	++	
FORWARD	->	= *i	*i =	++	==, !=
BIDIRECTIONAL		= *i	*i =	++, --	==, !=
RANDOM-ACCESS	->, []	= *i	*i =	++, --, +=, -=, +, -	==, !=, <, >, <=, >=

АДАПТЕРЫ ПОСЛЕДОВАТЕЛЬНЫХ КОНТЕЙНЕРОВ

Адаптеры последовательных контейнеров – это контейнеры, созданные на основе существующих контейнеров, но имеющие ограниченный функционал работы

queue

priority
queue

stack

АССОЦИАТИВНЫЕ КОНТЕЙНЕРЫ

обеспечивают быстрый доступ к данным за счет того, что они, как правило, построены на основе сбалансированных деревьев поиска

set

multiset

map

multimap

SET

```
#include <set>
```

```
std::set<char> a;  
a.insert('A');  
a.insert('B');  
a.insert('A');  
for (auto& str : a) {  
    std::cout << str << ' ' ;  
}
```

A B

```
set<int, greater<int> > s1;  
s1.insert(3);  
s1.insert(13);  
s1.insert(5);  
s1.insert(8);
```

```
for (auto i : s1) {  
    cout << i << ' ' ;  
}
```

13 8 5 3

элементы хранятся в отсортированном порядке

элементы имеют уникальные значения

значение элемента не может быть изменено
после его добавления

соответствует реализации двоичного дерева
поиска

значения не индексируются

MAP

```
map<string, int> mp;  
mp["one"] = 1;  
mp["two"] = 2;  
mp["three"] = 3;  
map<string, int>::iterator it = mp.begin();  
while (it != mp.end()) {  
    cout << "Key: " << it->first  
        << ", Value: " << it->second << endl;  
    ++it;  
}  
cout << "Size of map: " << mp.size() << endl;
```

```
Key: one, Value: 1  
Key: three, Value: 3  
Key: two, Value: 2  
Size of map: 3
```

```
#include <map>
```

```
begin()
```

```
end()
```

```
size()
```

```
max_size()
```

```
empty()
```

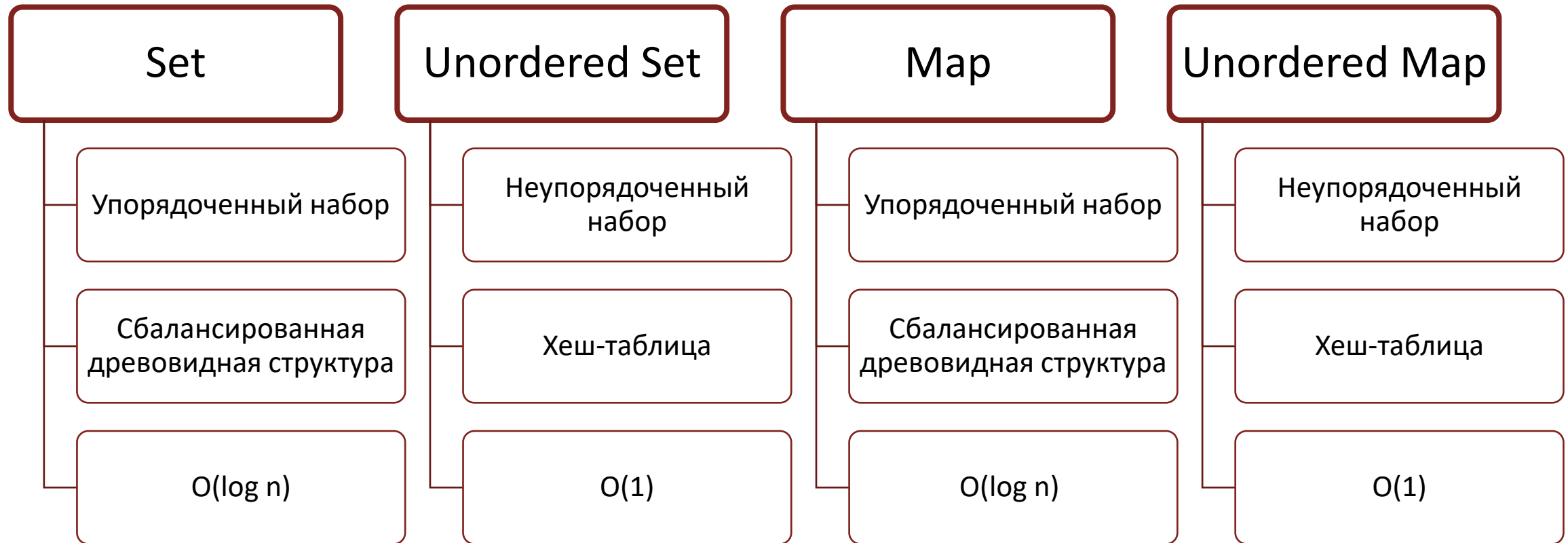
```
pair insert(keyvalue, mapvalue)
```

```
erase(iterator position)
```

```
erase(const g)
```

```
clear()
```

НЕУПОРЯДОЧЕННЫЕ АССОЦИАТИВНЫЕ КОНТЕЙНЕРЫ



ФУНКТОРЫ

```
int increment(int x) { return (x + 1); }
```

```
int arr[] = { 1, 2, 3, 4, 5 };  
int n = sizeof(arr) / sizeof(arr[0]);  
transform(arr, arr + n, arr, increment);  
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";
```

2 3 4 5 6

```
struct increment {  
    int num;  
    increment(int n) : num(n) {}  
    int operator () (int x) const {  
        return x + num;  
    }  
};
```

```
int arr[] = { 1, 2, 3, 4, 5 };  
int n = sizeof(arr) / sizeof(arr[0]);  
int to_add = 5;  
transform(arr, arr + n, arr, increment(to_add));  
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";
```

6 7 8 9 10

ПРЕДИКАТЫ

```
bool even(int k) { return k % 2 == 0; }

int main()
{
    list<int> L;
    int i, a;
    list<int>::iterator p;
    for (i = 1; i <= 10; i++)
        L.push_back(i);
    L.remove_if(even);
    for (p = L.begin(); p != L.end(); p++)
        cout << *p << " ";
    return 0;
}
```

Предикат – некоторая логическая функция, которая возвращает либо true, либо false для каждого элемента списка. Если для очередного элемента функция принимает значение true, то этот элемент будет отобран и с ним будет выполнено действие, иначе – нет

1 3 5 7 9

ПРЕДИКАТЫ

```
bool less1(int x, int y) {  
    return y < x;  
}
```

sort(less1)

L1.merge(L2, less1)

```
bool more(int x, int y) {  
    return x > y;  
}
```

unique(more)

СТАНДАРТНЫЕ ПРЕДИКАТЫ

equal_to	Бинарный	bool	$x == y$
not_equal_to	Бинарный	bool	$x != y$
greater	Бинарный	bool	$x > y$
less	Бинарный	bool	$x < y$
greater_equal	Бинарный	bool	$x \geq y$
less_equal	Бинарный	bool	$x \leq y$

название предиката <тип элементов> ()

```
L1.sort(greater <int> ());  
L2.sort(greater <int> ());  
L1.merge(L2, greater <int> ());
```

АЛГОРИТМЫ

```
#include <algorithm>
```

```
int arr[] = { 3, 8, 5, 13, 2 };  
sort(begin(arr), end(arr));  
for (int i : arr)  
    cout << i << " ";
```

2 3 5 8 13

Алгоритмы поиска и подсчета

Алгоритмы сортировки

Алгоритмы удаления и замены

АЛГОРИТМЫ ПОИСКА И ПОДСЧЕТА

```
int a;
list<int> note;
list<int>::iterator p, q, t;
for (int i = 1; i <= 10; i++)
{
    cin >> a;
    note.push_back(a);
}
p = ++note.begin();
q = --note.end();
t = find(p, q, 3);
cout << *t << " ";
```

```
find(begin, end, what)
adjacent_find(start, end)
search(start, end, sbegin, send)
count(sbegin, send, d)
count_if(sbegin, send, pred)
max_element(sbegin, send)
min_element(sbegin, send)
```

```
binary_search(begin, end, what)
```

1)	3	4	5	6	7	8	9	0	1	2,	результат	-	2
2)	4	5	3	6	7	8	3	0	1	2,	результат	-	3
3)	4	5	3	6	7	8	3	0	1	3,	результат	-	3

АЛГОРИТМЫ СОРТИРОВКИ

`sort(start,end)` – сортирует элементы диапазона `[start,end)` в порядке возрастания

`sort(start,end,pred)` – сортирует элементы диапазона `[start,end)` в порядке, заданном с помощью предиката `pred`

`reverse(start,end)` – инвертирует элементы последовательности `start-end`

`random_shuffle(start,end)` – сортирует элементы между `start` и `end` в случайном порядке

`is_sorted(start,end)` – возвращает логическое значение `true`, если элементы указанного диапазона отсортированы по возрастанию

`is_sorted(start,end,pred)` – возвращает логическое значение `true`, если элементы указанного диапазона отсортированы в соответствии с приведенной функцией-предикатом

АЛГОРИТМЫ УДАЛЕНИЯ И ЗАМЕНЫ

```
int a;
list<int> note1, note2;
list<int>::iterator p;
for (int i = 1; i <= 5; i++) {
    cin >> a;
    note1.push_back(a);
}
for (int i = 1; i <= 10; i++) {
    cin >> a;
    note2.push_back(a);
}
copy(note1.begin(), note1.end(), note2.begin());
for (p = note2.begin(); p != note2.end(); p++)
    cout << *p << " ";
```

```
remove (begin ,end ,what )
remove_if (begin ,end ,pred)
unique (begin ,end)
unique (begin ,end ,pred)
replace (begin ,end ,value1 ,value2 )
replace_if (begin ,end ,pred ,value)
fill (begin ,end ,value)
copy (begin ,end ,result )
```

Входные данные:

note1 - 1 1 1 1 1

note2 - 1 2 3 4 5 6 7 8 9 0

Результат: 1 1 1 1 1 6 7 8 9 0

ALGORITHM

```
int ar[5] = { 1, 2, -3, 4, 5 };
```

```
all_of(ar, ar + 5, [](int x) { return x > 0; }) ?  
    cout << "Все элементы положительные" :  
    cout << "Не все элементы положительные";  
any_of(ar, ar + 5, [](int x) { return x < 0; }) ?  
    cout << "Есть отрицательный элемент" :  
    cout << "Нет отрицательных элементов";  
none_of(ar, ar + 5, [](int x) { return x < 0; }) ?  
    cout << "Нет отрицательных элементов" :  
    cout << "Есть отрицательные элементы";
```

```
int ar_new[5];  
copy_n(ar, 5, ar_new);
```

```
int ar[5] = { 0 };  
iota(ar, ar + 5, 20);
```

